



King Saud University

College of Computer and Information Sciences

Department of Information Technology

IT486 Project

Name	ID
Joud Bakarman	

Supervised by:

Dr. Ameera Almasoud

Table of Contents

1. INTRODUCTION	3
2. DATASET DESCRIPTION.....	3
2.1 Data Size	3
2.2 Content.....	3
3. DATA PREPROCESSING.....	4
3.1 Read and Load Dataset	4
3.2 Data Cleaning	6
3.3 Data Transformation	8
3.4 Feature Selection	9
4. RDD OPERATIONS.....	11
4.1 Data preparation	11
4.2 RDD Transformation Operations	12
4.3 RDD Action Operations	15
5. SQL OPERATIONS	17
6. MACHINE LEARNING OPERATIONS.....	22
6.1 ML Problem	22
6.2 Importing Libraries and Preparing the labelled Data.....	23
6.3 Model Training	24
6.4 Model Evaluation and Results	25
7. CONCLUSION	27

1. Introduction

Smoke Detection and Environmental Monitoring aims to explore a dataset consisting of IoT sensor readings to understand the relationship between environmental factors and the occurrence of fire alarms. This analysis investigates various sensor data points such as temperature, humidity, air quality (TVOC, eCO₂), and particulate matter levels (PM_{1.0}, PM_{2.5}) to uncover patterns that could signal fire-related events. By applying data preparation methods and predictive modeling techniques, this study seeks to provide insights into how environmental conditions impact fire detection systems. The dataset used is representative of real-world sensor outputs, offering valuable information for safety professionals and organizations to identify trends and improve fire alarm accuracy. Data cleaning processes are employed to handle missing values and remove duplicates, ensuring the reliability of the analysis. Outliers are addressed with appropriate statistical techniques to maintain analysis integrity. The dataset is further processed to explore correlations between environmental variables and fire alarms. Machine learning algorithms are utilized to build predictive models, aiming for high accuracy and performance. This research offers recommendations for enhancing fire detection systems and provides insights into the environmental factors influencing fire safety technology.

2. Dataset Description

2.1 Data Size

This dataset contains 16 attributes and 62630 objects.

2.2 Content

The dataset consists of sensor readings related to smoke detection, including temperature, humidity, air quality, and particulate matter levels, along with a fire alarm indicator. It captures environmental data to monitor and detect smoke or fire-related events.

Columns include:

- **UTC**: Timestamp of the sensor readings.
- **Temperature[C]**: Recorded temperature in Celsius.

- **Humidity[%]**: Recorded humidity percentage.
- **TVOC[ppb]**: Total Volatile Organic Compounds concentration in parts per billion.
- **eCO2[ppm]**: Equivalent CO2 concentration in parts per million.
- **Raw H2**: Raw hydrogen sensor readings.
- **Raw Ethanol**: Raw ethanol sensor readings.
- **Pressure[hPa]**: Atmospheric pressure in hectopascals.
- **PM1.0**: Particulate matter concentration (1.0 μm) in micrograms per cubic meter.
- **PM2.5**: Particulate matter concentration (2.5 μm) in micrograms per cubic meter.
- **NC0.5, NC1.0, NC2.5**: Nanoclustering values for particulate matter at different sizes.
- **CNT**: Counter for data points.
- **Fire Alarm**: Indicator for whether a fire alarm was triggered (1 for triggered, 0 for not).

The dataset is provided in CSV format and contains sensor data used to monitor environmental factors that may correlate with smoke detection and fire alarms.

Source: <https://www.kaggle.com/datasets/deepcontractor/smoke-detection-dataset>

3. Data Preprocessing

3.1 Read and Load Dataset

This step is done using google colab tool, which used python programming language.

Import the necessary libraries:

```
[ ] import pandas as pd
import warnings
from scipy.stats import zscore
import numpy as np
from sklearn.neighbors import LocalOutlierFactor
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
import warnings
```

Figure 1: Import libraries

Read and prepare the dataset:

In the next section, I start by importing the necessary libraries such as pandas, NumPy, and others for data manipulation and statistical analysis. I then loaded the dataset

into a pandas DataFrame. To prepare the data for analysis, I created a copy of the DataFrame for preprocessing.

I removed the first column, which is the number of the row, and drop the 'UTC' column, as it's not needed for the analysis. Finally, I save the preprocessed dataset as 'dataset.csv' and display it to check the changes.

```
df = pd.read_csv('smoke_detection_iot.csv')

# Create a copy for preprocessing
df_preprocessed = df.copy()

# Remove the first column
df_preprocessed = df_preprocessed.iloc[:, 1:]

# Drop the 'UTC' column to exclude it from analysis
if 'UTC' in df_preprocessed.columns:
    df_preprocessed = df_preprocessed.drop(columns=['UTC'])

# Save the preprocessed dataset
df_preprocessed.to_csv("dataset.csv", index=False)
display(df_preprocessed)
print("Preprocessed dataset saved as 'Preprocessed_dataset.csv'.")
```

	Temperature[C]	Humidity[%]	TVOC[ppb]	eCO2[ppm]	Raw H2	Raw Ethanol	Pressure[hPa]	PM1.0	PM2.5	NC0.5	NC1.0	NC2.5	CNT	Fire Alarm
0	20.000	57.36	0	400	12306	18520	939.735	0.00	0.00	0.00	0.000	0.000	0	0
1	20.015	56.67	0	400	12345	18651	939.744	0.00	0.00	0.00	0.000	0.000	1	0
2	20.029	55.96	0	400	12374	18764	939.738	0.00	0.00	0.00	0.000	0.000	2	0
3	20.044	55.28	0	400	12390	18849	939.736	0.00	0.00	0.00	0.000	0.000	3	0
4	20.059	54.69	0	400	12403	18921	939.744	0.00	0.00	0.00	0.000	0.000	4	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
62625	18.438	15.79	625	400	13723	20569	936.670	0.63	0.65	4.32	0.673	0.015	5739	0
62626	18.653	15.87	612	400	13731	20588	936.678	0.61	0.63	4.18	0.652	0.015	5740	0
62627	18.867	15.84	627	400	13725	20582	936.687	0.57	0.60	3.95	0.617	0.014	5741	0

Figure 2: Prepare the dataset

In this section, I first calculate the number of attributes in the preprocessed dataset. I then retrieve the data types of each attribute.

Next, I determine the total number of objects (rows) in the dataset. Finally, I print out the number of attributes, the data types of each attribute, and the number of objects in the dataset.

```

num_attributes = len(df_preprocessed.columns)
attribute_types = df_preprocessed.dtypes.to_frame().rename(columns={0: 'Data Types'})
num_objects = len(df_preprocessed)
class_name = df_preprocessed.columns[-1]
print("\033[1mNumber of attributes:\033[0m", num_attributes)
print("\n")
print("\033[1mAttribute types:\033[0m")
print(attribute_types)
print("\n")
print("\033[1mNumber of objects:\033[0m", num_objects)
print("\n")

```

Number of attributes: 14

Attribute types:

	Data Types
Temperature[C]	float64
Humidity[%]	float64
TVOC[ppb]	int64
eCO2[ppm]	int64
Raw H2	int64
Pressure[hPa]	float64
PM1.0	float64
PM2.5	float64
NC0.5	float64
NC1.0	float64
NC2.5	float64
CNT	int64
Fire Alarm	int64

Number of objects: 62630

Figure 3: Data attributes and objects

3.2 Data Cleaning

3.2.1 Duplicates

In this section, I first check for any duplicate rows in the preprocessed dataset using the `duplicated()` function and count the number of duplicates with `.sum()`. I then print the total number of duplicates.

Next, I remove the duplicate rows from the dataset using the `drop_duplicates()` function. After removing the duplicates, I check again for any remaining duplicates and print the new count of duplicates. This ensures that the dataset no longer contains any duplicate entries.

This code confirms that the dataset has no rows that are identical copies of other rows based on all columns.

```

[5] duplicates_sum = df_preprocessed.duplicated().sum()
    print("Number of duplicates: ");
    print(duplicates_sum);

```

Number of duplicates:
2

Dropping the duplicates:

```

[6] df_preprocessed = df_preprocessed.drop_duplicates()
    duplicates_sum = df_preprocessed.duplicated().sum()
    print("Duplicates removed. New shape of DataFrame:", df_preprocessed.shape)
    print("Number of duplicates: ");
    print(duplicates_sum);

```

Duplicates removed. New shape of DataFrame: (62628, 14)
Number of duplicates:
0

Figure 4: Duplicates detection and removal

3.2.2 Missing Values

In this section, I check for missing values in the preprocessed dataset by using the `isna()` function. I then calculate the number of missing values in each column by applying `.sum()` to the result.

This gives the total count of missing values for each column, which is then printed and indicated that there are no missing values in the dataset.

```
[7] missing_values = df_preprocessed.isna()
missing_counts = missing_values.sum();
print(missing_counts)
```

Temperature[C]	0
Humidity[%]	0
TVOC[ppb]	0
eCO2[ppm]	0
Raw H2	0
Raw Ethanol	0
Pressure[hPa]	0
PM1.0	0
PM2.5	0
NC0.5	0
NC1.0	0
NC2.5	0
CNT	0
Fire Alarm	0

dtype: int64

Figure 5: Missing values detection

3.2.3 Outlier Detection

In this section, I define a function `remove_outlier()` to calculate the IQR and determine the lower and upper boundaries for detecting outliers. The dataset is split into two subsets based on the 'Fire Alarm' values (0.0 and 1.0), and outliers are removed from each subset separately. Extreme outliers are adjusted by selecting the 70th percentile for the upper range and the 30th percentile for the lower range. After removing outliers, the cleaned dataset is saved to 'dataset.csv', and the shape of the dataset is displayed. after removing outliers, the dataset still contains 53,755 rows and 14 attributes.

```

Outlier Detection:

[49] # Function to remove outliers using IQR
def remove_outlier(col):
    Q1, Q3 = np.percentile(col, [25, 75])
    IQR = Q3 - Q1
    lower_range = Q1 - (1.5 * IQR)
    upper_range = Q3 + (1.5 * IQR)
    return lower_range, upper_range

# Separate each class
df0 = df_preprocessed[df_preprocessed['Fire Alarm'] == 0.0]
df1 = df_preprocessed[df_preprocessed['Fire Alarm'] == 1.0]

# Dictionary to store outlier details
dec = {"Column": [], "outliers": [], "u_range": [], "l_range": [], "upper": [], "lower": [],
       "Nu_range": [], "Nl_range": [], "N_upper": [], "N_lower": []}

# Iterate through all feature columns except 'Fire Alarm'
for column in df_preprocessed.drop(columns=['Fire Alarm']).columns:
    # Process for Fire Alarm = 0
    lr, ur = remove_outlier(df0[column])
    u_data = df_preprocessed[(df_preprocessed['Fire Alarm'] == 0) & (df_preprocessed[column] > ur)][column]
    l_data = df_preprocessed[(df_preprocessed['Fire Alarm'] == 0) & (df_preprocessed[column] < lr)][column]

    # Adjust extreme outliers
    if not u_data.empty:
        u_data = sorted(u_data)
        index = int(round(len(u_data) * 0.7, 0))
        ur = u_data[index]

    if not l_data.empty:
        l_data = sorted(l_data)
        index = int(round(len(l_data) * 0.3, 0))
        lr = l_data[index]

    index = df_preprocessed[(df_preprocessed['Fire Alarm'] == 0) & ((df_preprocessed[column] < lr) | (df_preprocessed[column] > ur))].index
    df_preprocessed.drop(index, inplace=True)

    # Process for Fire Alarm = 1
    lr, ur = remove_outlier(df1[column])
    u_data = df_preprocessed[(df_preprocessed['Fire Alarm'] == 1) & (df_preprocessed[column] > ur)][column]
    l_data = df_preprocessed[(df_preprocessed['Fire Alarm'] == 1) & (df_preprocessed[column] < lr)][column]

    if not u_data.empty:
        u_data = sorted(u_data)
        index = int(round(len(u_data) * 0.7, 0))
        ur = u_data[index]

    if not l_data.empty:
        l_data = sorted(l_data)
        index = int(round(len(l_data) * 0.3, 0))
        lr = l_data[index]

    index = df_preprocessed[(df_preprocessed['Fire Alarm'] == 1) & ((df_preprocessed[column] < lr) | (df_preprocessed[column] > ur))].index
    df_preprocessed.drop(index, inplace=True)

[51] df_preprocessed.to_csv('dataset.csv', index=False)
print("Outlier removal complete.")
df_preprocessed.shape

Outlier removal complete.
(53755, 14)

```

Figure 6: Outlier detection and removal

3.3 Data Transformation

3.3.1 Normalization

In this section, I initialized a MinMaxScaler to scale the features of the dataset. I apply the scaler to all columns except 'Fire Alarm' (the class label).

The fit_transform() method normalizes these columns to a range between 0 and 1. After scaling the features, I save the normalized dataset to 'dataset.csv' and display the scaled columns.

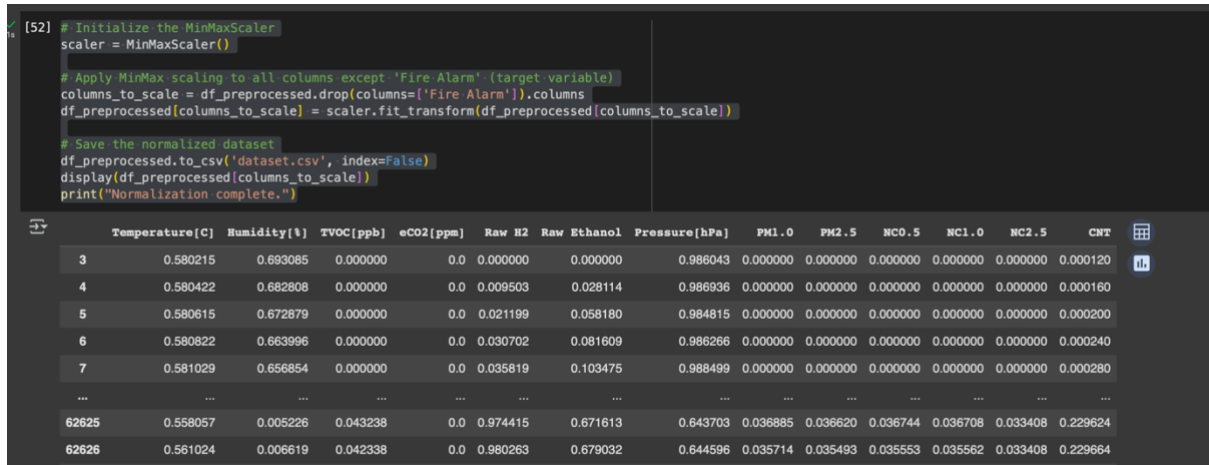


Figure 7: Data normalization

3.4 Feature Selection

In this section, I calculate the correlation between each feature and the class label 'Fire Alarm'. I create a DataFrame to visualize the correlations and sort the features by their correlation. I then plot the correlations to identify the "elbow point" for feature selection. From the plot, I determined that 6 features should be chosen for the model.

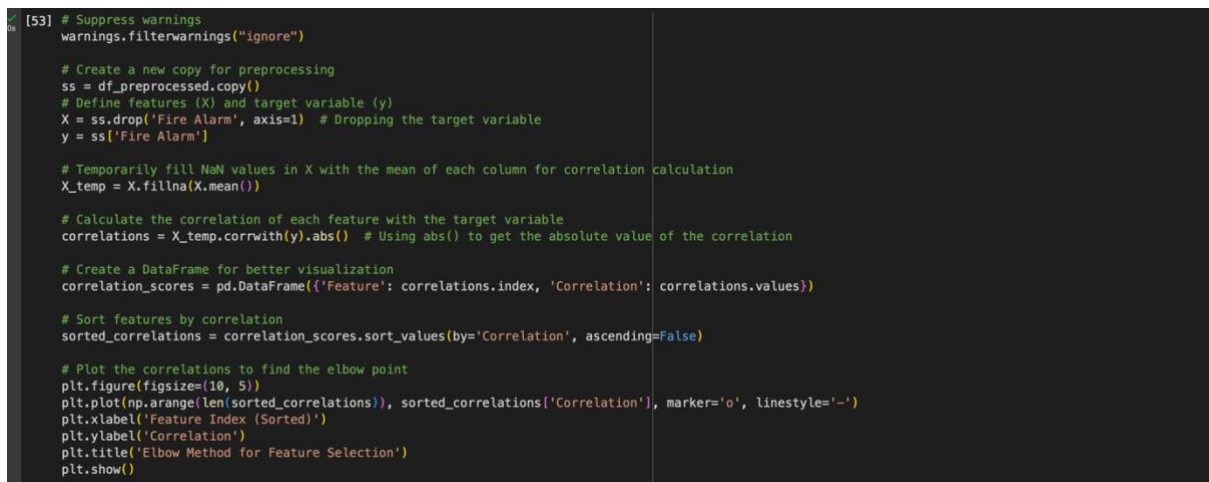


Figure 8: Correlation calculation

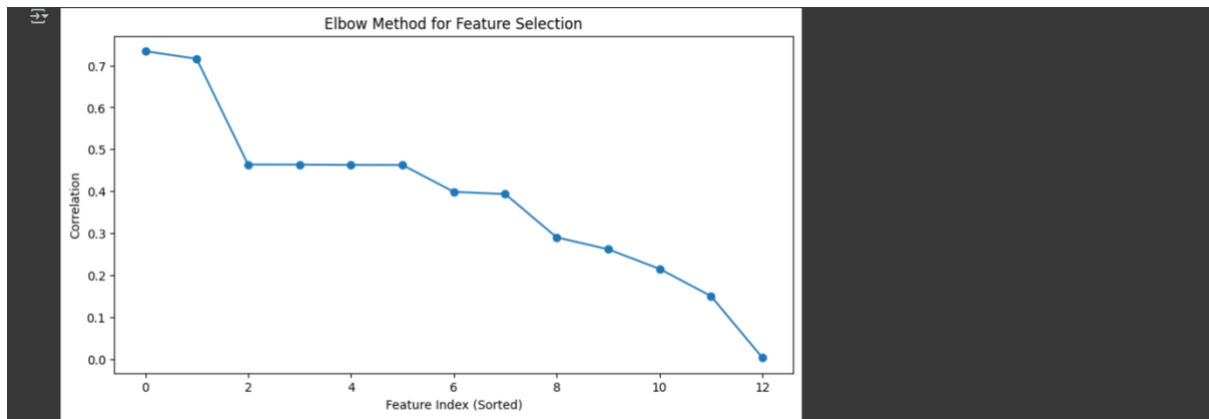


Figure 9: Elbow method visualization

Next, I sort the features by their absolute correlation and select the top 6 features with the highest correlation to the class label, 'Fire Alarm'. The top 6 features identified are **CNT**, **Raw Ethanol**, **NC0.5**, **PM1.0**, **PM2.5**, and **NC1.0**. I then plot these top 6 features to visually represent their correlation.

The plot is displayed with the most important feature at the top. Afterward, I keep only the selected top 6 features along with the target variable in the DataFrame and save the updated dataset as 'Preprocessed_dataset.csv' for further analysis.

```
# Sort the features by their absolute correlation and get the top 6
top_6_features = correlation_scores.nlargest(6, 'Correlation')['Feature'].tolist()

# Plot the top 6 features
plt.figure(figsize=(10, 6))
plt.barh(top_6_features, correlation_scores.set_index('Feature').loc[top_6_features]['Correlation'], color='mediumslateblue')
plt.xlabel('Correlation')
plt.ylabel('Feature')
plt.title('Top 6 Features by Correlation with Fire Alarm')
plt.gca().invert_yaxis() # Invert y-axis to have the most important feature at the top
plt.show()

# Keep only the top 6 features and the target variable in the DataFrame
ss = ss[top_6_features + ['Fire Alarm']]

# Save the updated DataFrame
ss.to_csv('Preprocessed_dataset.csv', index=False)

print("Feature selection complete.")
```

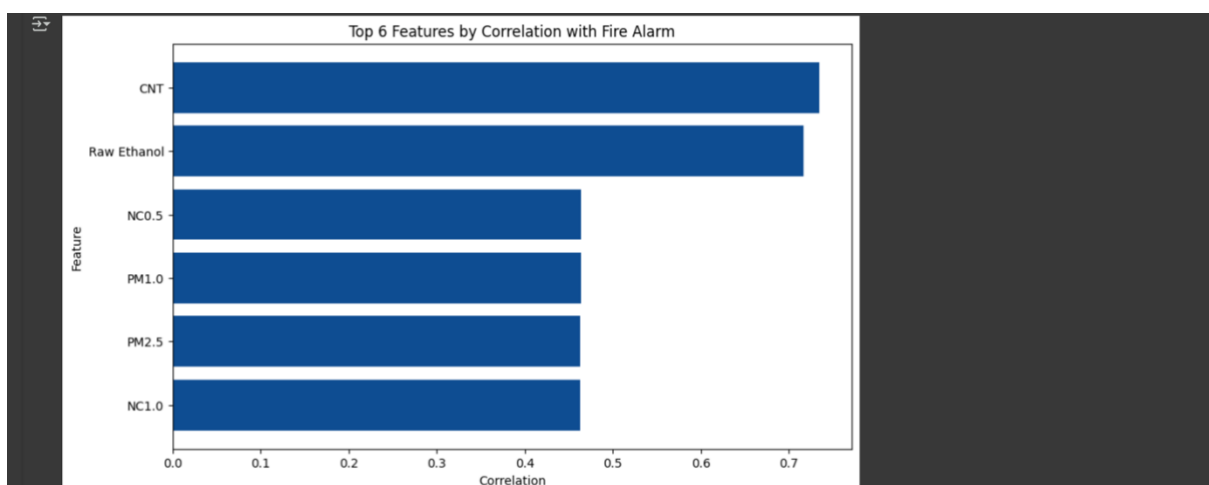
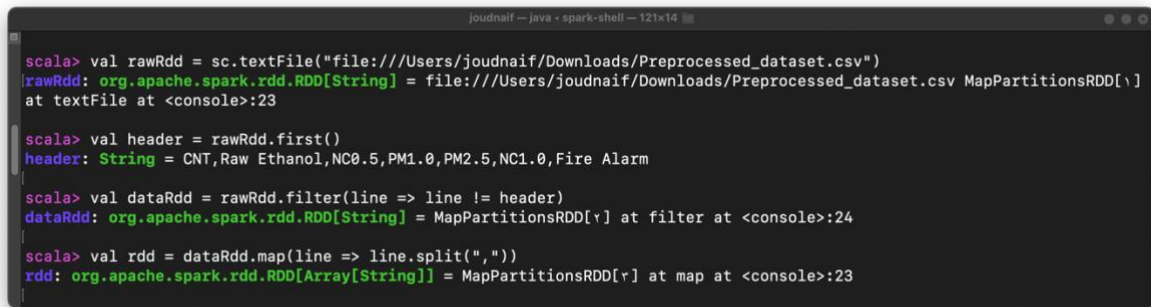


Figure 10: Feature selection

4. RDD Operations

4.1 Data preparation

In this section I started loading, cleaning, and preparing the dataset for further analysis using Apache Spark in Scala.



```
joudnaif -- java - spark-shell -- 121x14
scala> val rawRdd = sc.textFile("file:///Users/joudnaif/Downloads/Preprocessed_dataset.csv")
rawRdd: org.apache.spark.rdd.RDD[String] = file:///Users/joudnaif/Downloads/Preprocessed_dataset.csv MapPartitionsRDD[1]
at textFile at <console>:23

scala> val header = rawRdd.first()
header: String = CNT,Raw Ethanol,NC0.5,PM1.0,PM2.5,NC1.0,Fire Alarm

scala> val dataRdd = rawRdd.filter(line => line != header)
dataRdd: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[2] at filter at <console>:24

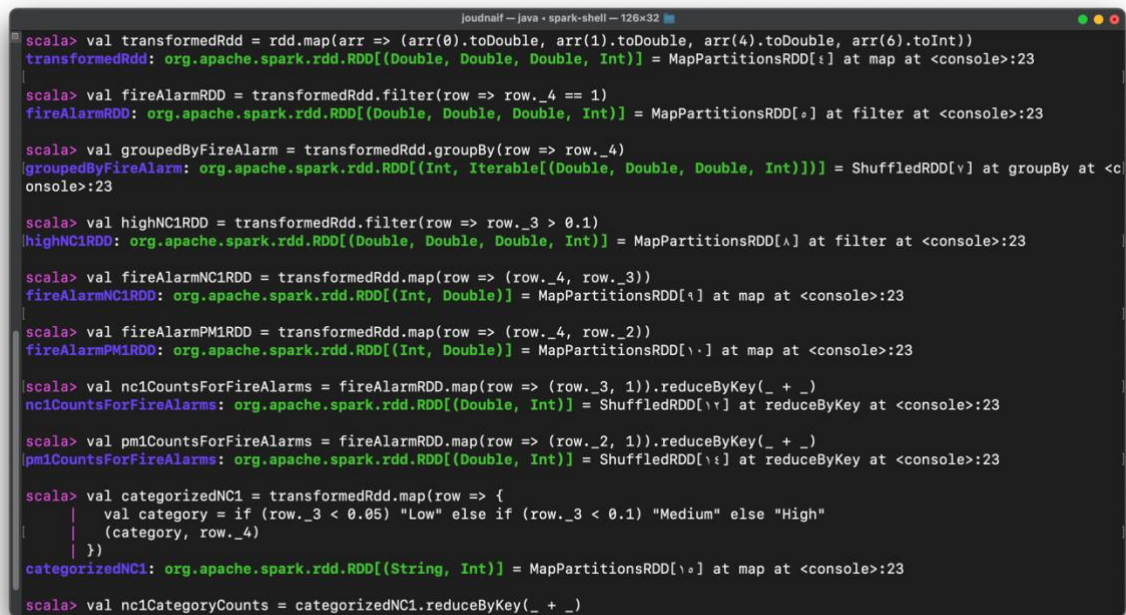
scala> val rdd = dataRdd.map(line => line.split(","))
rdd: org.apache.spark.rdd.RDD[Array[String]] = MapPartitionsRDD[3] at map at <console>:23
```

Figure 11: Data loading and preparing

First, the CSV file is read into an RDD (rawRdd), where each row is treated as a string. Since the first row contains column headers, it is extracted separately using `.first()`. To ensure that only actual data is processed, a filtered RDD (dataRdd) is created by removing the header row.

Finally, the dataset is transformed into an RDD of arrays (rdd), where each row is split by commas, making it easier to work with structured data in subsequent transformations and actions. This step ensures the dataset is clean and ready for further computations, such as filtering, mapping, and aggregations.

4.2 RDD Transformation Operations



```
scala> val transformedRdd = rdd.map(arr => (arr(0).toDouble, arr(1).toDouble, arr(4).toDouble, arr(6).toInt))
transformedRdd: org.apache.spark.rdd.RDD[(Double, Double, Double, Int)] = MapPartitionsRDD[1] at map at <console>:23

scala> val fireAlarmRDD = transformedRdd.filter(row => row._4 == 1)
fireAlarmRDD: org.apache.spark.rdd.RDD[(Double, Double, Double, Int)] = MapPartitionsRDD[1] at filter at <console>:23

scala> val groupedByFireAlarm = transformedRdd.groupBy(row => row._4)
groupedByFireAlarm: org.apache.spark.rdd.RDD[(Int, Iterable[(Double, Double, Double, Int)])] = ShuffledRDD[1] at groupBy at <console>:23

scala> val highNC1RDD = transformedRdd.filter(row => row._3 > 0.1)
highNC1RDD: org.apache.spark.rdd.RDD[(Double, Double, Double, Int)] = MapPartitionsRDD[1] at filter at <console>:23

scala> val fireAlarmNC1RDD = transformedRdd.map(row => (row._4, row._3))
fireAlarmNC1RDD: org.apache.spark.rdd.RDD[(Int, Double)] = MapPartitionsRDD[1] at map at <console>:23

scala> val fireAlarmPM1RDD = transformedRdd.map(row => (row._4, row._2))
fireAlarmPM1RDD: org.apache.spark.rdd.RDD[(Int, Double)] = MapPartitionsRDD[1] at map at <console>:23

scala> val nc1CountsForFireAlarms = fireAlarmRDD.map(row => (row._3, 1)).reduceByKey(_ + _)
nc1CountsForFireAlarms: org.apache.spark.rdd.RDD[(Double, Int)] = ShuffledRDD[1] at reduceByKey at <console>:23

scala> val pm1CountsForFireAlarms = fireAlarmRDD.map(row => (row._2, 1)).reduceByKey(_ + _)
pm1CountsForFireAlarms: org.apache.spark.rdd.RDD[(Double, Int)] = ShuffledRDD[1] at reduceByKey at <console>:23

scala> val categorizedNC1 = transformedRdd.map(row => {
  |   val category = if (row._3 < 0.05) "Low" else if (row._3 < 0.1) "Medium" else "High"
  |   (category, row._4)
  | })
categorizedNC1: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[1] at map at <console>:23

scala> val nc1CategoryCounts = categorizedNC1.reduceByKey(_ + _)
```

Figure 12: RDD transformation operations

I chose these 10 transformation operations:

1- Converting Dataset to Structured Format

This transformation maps each row of the dataset to a structured format with selected numerical columns.

This step ensures that numerical values are properly formatted for further processing and analysis, making computations more efficient and structured.

2- Filtering Data by Fire Alarm Activation

In this step, the dataset is filtered to keep only rows where the Fire Alarm was triggered (`_4 == 1`). This transformation creates a new RDD (`fireAlarmRDD`) that contains only instances where a fire event was detected.

Helps focus analysis on cases where fire alarms were activated, allowing further investigation into contributing factors like NC1.0 and PM1.0 levels.

3- Grouping Data by Fire Alarm Status

The dataset is grouped based on whether the Fire Alarm was triggered (**0 = No Alarm, 1 = Alarm**). The result (`groupedByFireAlarm`) clusters records under each category, making it possible to compare alarm-triggered and non-alarm cases.

This helps in analyzing the differences in sensor readings between instances when a fire alarm is activated and when it is not.

4- Filtering Data for High NC1.0 Levels

This transformation filters the dataset to include only rows where NC1.0 (particle concentration) is greater than 0.1. The resulting RDD (`highNC1RDD`) contains instances where high airborne particles were detected, which could indicate potential fire conditions.

It allows analysis of high-pollution scenarios and their correlation with fire alarm activation.

5- Mapping Fire Alarm Status to NC1.0 Values

Each row is transformed into a (Fire Alarm Status, NC1.0 Level) pair in `fireAlarmNC1RDD`. This transformation simplifies the dataset by directly linking NC1.0 levels with Fire Alarm occurrences.

It helps in determining if specific NC1.0 thresholds are more likely to trigger alarms.

6- Mapping Fire Alarm Status to PM1.0 Values

Similar to the previous step, this transformation maps each row into a (Fire Alarm Status, PM1.0 Level) pair in `fireAlarmPM1RDD`. This transformation allows for a direct comparison of PM1.0 levels in fire alarm vs. non-fire alarm cases.

It helps determine if PM1.0 levels are a contributing factor in fire alarm detection.

7- Counting Fire Alarms for Each NC1.0 Value

This transformation groups NC1.0 values when a fire alarm was triggered and counts how many times each value appears. The result (`nc1CountsForFireAlarms`) provides a frequency distribution of NC1.0 levels during fire events.

It is useful for identifying which NC1.0 values are most frequently associated with fire alarms.

8- Counting Fire Alarms for Each PM1.0 Value

Similar to the NC1.0 count, this transformation creates a frequency distribution of PM1.0 values that appeared when Fire Alarms were triggered (`pm1CountsForFireAlarms`).

This helps analyze whether specific PM1.0 levels consistently appear during fire alarm events.

9- Categorizing NC1.0 Levels into Ranges

This transformation classifies NC1.0 values into three categories:

- "Low" if $NC1.0 < 0.05$
- "Medium" if NC1.0 is between 0.05 and 0.1
- "High" if $NC1.0 > 0.1$

The result (`categorizedNC1`) helps segment the dataset based on airborne particle concentrations.

It simplifies the analysis of NC1.0 levels and makes it easier to identify patterns in alarm activation.

10- Counting Fire Alarms in NC1.0 Categories

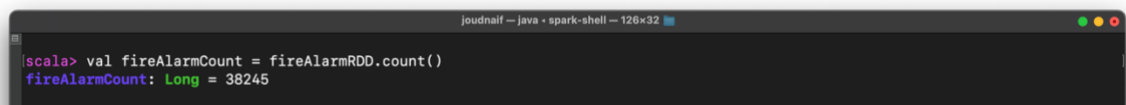
This transformation calculates how many Fire Alarms occurred within each NC1.0 category (`nc1CategoryCounts`).

It helps identify which NC1.0 category (Low, Medium, or High) is most commonly associated with Fire Alarms, which is useful for setting alarm thresholds.

4.3 RDD Action Operations

1- Frequency of Fire Alarm Activations

Out of 53,756 total records, 38,245 resulted in fire alarm activations, making up 71.1% of the dataset. This high activation rate suggests that the system frequently detects conditions that meet the alarm criteria. While this could indicate effective sensitivity, it also raises concerns about the possibility of false alarms due to overly sensitive detection thresholds.



```
joudnaif -- java - spark-shell -- 128x32
scala> val fireAlarmCount = fireAlarmRDD.count()
fireAlarmCount: Long = 38245
```

2- NC1.0 Levels During Fire Alarms

The average NC1.0 concentration during fire alarms is 0.11, with a maximum of 0.176 and a minimum of 0.03. The fact that alarms still activate at low NC1.0 values suggests that while higher NC1.0 levels contribute to alarm triggers, they are not the sole determining factor. Other pollutants or environmental conditions likely influence activation.



```
scala> val avgNC1WithFire = fireAlarmRDD.map(_._3).sum() / fireAlarmRDD.count()
avgNC1WithFire: Double = 0.11008656935730998
```

3- Maximum NC1.0 When Fire Alarm Was Triggered

The highest NC1.0 recorded during fire alarms is 0.176. This represents an extreme case of particle pollution when an alarm was activated, highlighting conditions under which the system considers the environment dangerous.



```
scala> val maxNC1WithFire = fireAlarmRDD.map(_._3).max()
maxNC1WithFire: Double = 0.1763380281690141
```

4- Maximum PM1.0 When Fire Alarm Was Triggered

The highest PM1.0 level recorded during fire alarms is 0.86. This suggests that extremely high PM1.0 levels can be a strong fire indicator. However, since the lowest PM1.0 recorded during alarms is significantly lower, the system does not require PM1.0 to reach extreme values to trigger an alarm.

```
scala> val maxPM1WithFire = fireAlarmRDD.map(_._2).max()
maxPM1WithFire: Double = 0.8609918000780938
```

5- Minimum PM1.0 When Fire Alarm Was Triggered

The lowest PM1.0 recorded during alarms is 0.198. This suggests that while PM1.0 plays a role in fire detection, alarms can still activate even when PM1.0 levels are relatively low, reinforcing the idea that multiple factors influence the system.

```
scala> val minPM1WithFire = fireAlarmRDD.map(_._2).min()
minPM1WithFire: Double = 0.19836001561889915
```

6- Minimum NC1.0 When Fire Alarm Was Triggered

The lowest NC1.0 recorded during alarms is 0.03. Since this value is relatively low, it means that alarms are still triggered at small concentrations of airborne particles. This suggests that additional factors, such as ethanol concentration or sudden changes in air quality, may contribute to activation.

```
scala> val minNC1WithFire = fireAlarmRDD.map(_._3).min()
minNC1WithFire: Double = 0.030422535211267608
```

7- Fire Alarms in NC1.0 Categories

Among all fire alarms, 27,856 cases (72.8%) occurred when NC1.0 was high, 8,861 (23.2%) when it was medium, and only 1,528 (4%) when it was low. This distribution suggests that high NC1.0 levels are a major factor in alarm activation, while alarms occurring at low NC1.0 levels are relatively rare.

```
scala> val fireAlarmsByNC1Category = categorizedNC1.filter(_._2 == 1).countByKey()
fireAlarmsByNC1Category: scala.collection.Map[String,Long] = Map(High -> 27856, Low -> 1528, Medium -> 8861)
```

8- Overall Average PM1.0 in the Dataset

The dataset's overall PM1.0 average is 0.369, which provides a benchmark for assessing fire alarm activation conditions. Since a significant number of fire alarms occurred at lower PM1.0 levels, it suggests that while PM1.0 is a contributing factor, it is not the sole determinant.

```
scala> val avgPM1Overall = transformedRdd.map(_._2).sum() / transformedRdd.count()
avgPM1Overall: Double = 0.3691538950763994
```


9- Fire Alarms Triggered When PM1.0 Exceeds Average

A total of 7,861 fire alarms (20.5%) occurred when PM1.0 was higher than the dataset's average of 0.369. This suggests that while elevated PM1.0 increases the likelihood of alarm activation, most alarms still occur at lower PM1.0 levels, meaning other factors contribute significantly.

```
scala> val fireAlarmsWithHighPM1 = fireAlarmRDD.filter(row => row._2 > avgPM10Overall).count()
fireAlarmsWithHighPM1: Long = 7861
```

10- Fire Alarms When NC1.0 and PM1.0 Were High

A total of 27,856 fire alarms were triggered when NC1.0 was greater than 0.1 and PM1.0 was also above 0.1. This shows that a large portion of alarms occurred when both particle concentrations were elevated, reinforcing the idea that high NC1.0 and PM1.0 together strongly contribute to fire alarm activations.

```
scala> val fireAlarmsWithHighNC1AndPM1 = fireAlarmRDD.filter(row => row._3 > 0.1 && row._2 > 0.1).count()
fireAlarmsWithHighNC1AndPM1: Long = 27856
```

5. SQL Operations

To explore the fire alarm dataset using SQL queries, I began by importing the necessary Spark libraries and initializing a SQLContext. The dataset was already cleaned and transformed into an RDD, and I proceeded to define a schema programmatically to ensure accurate column data types. The schema included fields such as CNT, Raw Ethanol, NC0.5, PM1.0, PM2.5, NC1.0, and Fire Alarm. I then converted the RDD into a DataFrame and registered it as a temporary view named `fire_alarm_data`, allowing us to perform SQL queries across the entire dataset of 53,756 records.

```
joudnaif — java · spark-shell — 128x31

scala> import org.apache.spark.sql.SQLContext
import org.apache.spark.sql.SQLContext

scala> val sqlContext = new SQLContext(sc)
warning: one deprecation (since 2.0.0); for details, enable `:setting -deprecation` or `:replay -deprecation`
sqlContext: org.apache.spark.sql.SQLContext = org.apache.spark.sql.SQLContext@671f1e0a

scala> import org.apache.spark.sql.types._
import org.apache.spark.sql.types._

scala> import org.apache.spark.sql.Row
import org.apache.spark.sql.Row

scala> val schema = StructType(Seq(
  |   StructField("CNT", DoubleType, true),
  |   StructField("Raw_Ethanol", DoubleType, true),
  |   StructField("NC0_5", DoubleType, true),
  |   StructField("PM1_0", DoubleType, true),
  |   StructField("PM2_5", DoubleType, true),
  |   StructField("NC1_0", DoubleType, true),
  |   StructField("Fire_Alarm", IntegerType, true)
  | ))
schema: org.apache.spark.sql.types.StructType = StructType(StructField(CNT,DoubleType,true),StructField(Raw_Ethanol,DoubleType,true),StructField(NC0_5,DoubleType,true),StructField(PM1_0,DoubleType,true),StructField(PM2_5,DoubleType,true),StructField(NC1_0,DoubleType,true),StructField(Fire_Alarm,IntegerType,true))

scala> val fireAlarmDF = sqlContext.createDataFrame(
  |   transformedRdd.map(Row.fromTuple), schema
  | )
fireAlarmDF: org.apache.spark.sql.DataFrame = [CNT: double, Raw_Ethanol: double ... 5 more fields]
```

Figure 13: Data preparation

5.1 Total Fire Alarm Activations

The first SQL query focused on identifying how many times the fire alarm was triggered. The result showed that 38,245 records had the alarm activated, accounting for 71.1% of the dataset. This indicates that the system frequently detects potential hazards, though the high percentage also suggests a need to evaluate whether some triggers might be false or overly sensitive.

5.2 Average NC1.0 and PM2.5 During Fire Alarms

This query calculated the average NC1.0 and PM2.5 levels when the alarm was active. The average NC1.0 was 0.11, and PM2.5 was also relatively elevated. These results indicate a strong association between particulate matter concentrations and alarm activation, reinforcing the importance of air quality in fire detection.

```
joudnaif - java - spark-shell -- 128x33

scala> fireAlarmDF.createOrReplaceTempView("fire_alarm_data")

scala> val totalFireAlarms = sqlContext.sql("SELECT COUNT(*) AS Fire_Alarm_Count FROM fire_alarm_data WHERE Fire_Alarm = 1")
totalFireAlarms: org.apache.spark.sql.DataFrame = [Fire_Alarm_Count: bigint]

scala> totalFireAlarms.show()
+-----+
|Fire_Alarm_Count|
+-----+
|          38245|
+-----+

scala> val avgNC1PM25WithFire = sqlContext.sql("SELECT AVG(NC1_0) AS Avg_NC1_0, AVG(PM2_5) AS Avg_PM2_5 FROM fire_alarm_data WHERE Fire_Alarm = 1")
avgNC1PM25WithFire: org.apache.spark.sql.DataFrame = [Avg_NC1_0: double, Avg_PM2_5: double]

scala> avgNC1PM25WithFire.show()
+-----+-----+
|Avg_NC1_0|Avg_PM2_5|
+-----+-----+
|0.11009924601370219|0.11008656935730998|
+-----+-----+

scala> val pm10Stats = sqlContext.sql("SELECT MAX(PM1_0) AS Max_PM1_0, MIN(PM1_0) AS Min_PM1_0 FROM fire_alarm_data WHERE Fire_Alarm = 1")
pm10Stats: org.apache.spark.sql.DataFrame = [Max_PM1_0: double, Min_PM1_0: double]
```

Figure 14: 5.1 & 5.2 Operations

5.3 Maximum and Minimum PM1.0 During Fire Alarms

The third operation identified the range of PM1.0 values associated with fire alarms. The maximum PM1.0 observed was 0.86 and the minimum was 0.198. This wide range suggests that while high PM1.0 levels are a significant factor, the system can also trigger alarms at lower thresholds, hinting at a multi-factor detection logic.

```
joudnaif - java - spark-shell -- 128x7

scala> pm10Stats.show()
+-----+-----+
|Max_PM1_0|Min_PM1_0|
+-----+-----+
|0.1762295081967213|0.030444964871194382|
+-----+-----+
```

Figure 15: 5.3 Operation

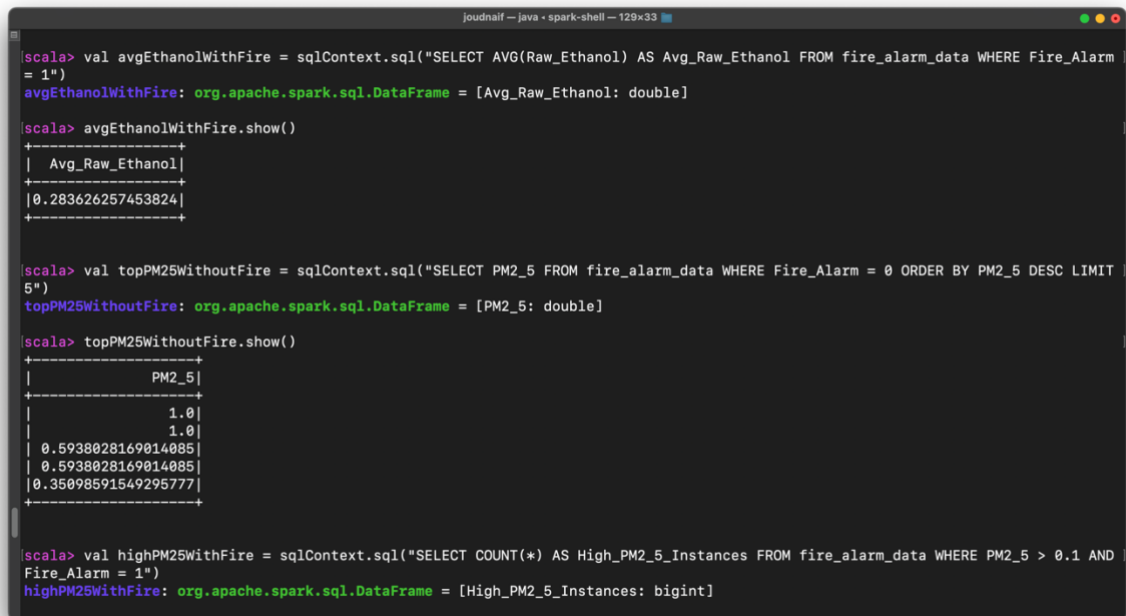
5.4 Average Raw Ethanol During Fire Alarms

I calculated the average raw ethanol value when the alarm was on, which came out to 0.28. This value is moderate and shows that while ethanol might contribute to detection, it likely plays a secondary role compared to particle-based pollutants like NC1.0 and PM values.

5.5 Top PM2.5 Values Without Fire Alarms

To investigate potential gaps in detection, I queried the top five PM2.5 readings that occurred without triggering a fire alarm. The highest value was 1.01, showing that even with very high

PM2.5 levels, alarms did not always activate. This suggests that PM2.5 alone is not sufficient for triggering the system, supporting the idea that multiple pollutants must rise simultaneously.



```
joudnaif — java · spark-shell — 129x33

[scala] val avgEthanolWithFire = sqlContext.sql("SELECT AVG(Raw_Ethanol) AS Avg_Raw_Ethanol FROM fire_alarm_data WHERE Fire_Alarm = 1")
avgEthanolWithFire: org.apache.spark.sql.DataFrame = [Avg_Raw_Ethanol: double]

[scala] avgEthanolWithFire.show()
+-----+
| Avg_Raw_Ethanol |
+-----+
| 0.283626257453824 |
+-----+

[scala] val topPM25WithoutFire = sqlContext.sql("SELECT PM2_5 FROM fire_alarm_data WHERE Fire_Alarm = 0 ORDER BY PM2_5 DESC LIMIT 5")
topPM25WithoutFire: org.apache.spark.sql.DataFrame = [PM2_5: double]

[scala] topPM25WithoutFire.show()
+-----+
| PM2_5 |
+-----+
| 1.0 |
| 1.0 |
| 0.5938028169014085 |
| 0.5938028169014085 |
| 0.35098591549295777 |
+-----+

[scala] val highPM25WithFire = sqlContext.sql("SELECT COUNT(*) AS High_PM2_5_Instances FROM fire_alarm_data WHERE PM2_5 > 0.1 AND Fire_Alarm = 1")
highPM25WithFire: org.apache.spark.sql.DataFrame = [High_PM2_5_Instances: bigint]
```

Figure 16: 5.4 & 5.5 Operations

5.6 Fire Alarms with PM2.5 Above 0.1

I counted how many alarms occurred when PM2.5 was greater than 0.1. A total of 27,856 records matched this condition. This indicates that more than 72% of fire alarms occurred under significantly elevated PM2.5 conditions, supporting its role as a strong contributor to alarm triggers.

5.7 Correlation Between NC1.0 and Fire Alarm

This more advanced query computed the correlation between NC1.0 values and fire alarm status. The result was 0.46, indicating a moderate positive correlation. This confirms that higher NC1.0 values increase the likelihood of an alarm being triggered, but other variables also play an important role.

```

scala> highPM25WithFire.show()
+-----+
|High_PM2_5_Instances|
+-----+
|                27856|
+-----+

scala> val nc1FireAlarmCorrelation = sqlContext.sql(
  """SELECT
    SUM((NC1_0 - (SELECT AVG(NC1_0) FROM fire_alarm_data)) * (Fire_Alarm - (SELECT AVG(Fire_Alarm) FROM fire_alarm_data))) /
    ((COUNT(*) * STDDEV(NC1_0) * STDDEV(Fire_Alarm))) AS Correlation
  FROM fire_alarm_data"""
)
nc1FireAlarmCorrelation: org.apache.spark.sql.DataFrame = [Correlation: double]

scala> nc1FireAlarmCorrelation.show()
+-----+
|Correlation|
+-----+
|0.46252387033987535|
+-----+

scala> val mostCommonNC05 = sqlContext.sql("SELECT NC0_5, COUNT(*) AS Count FROM fire_alarm_data WHERE Fire_Alarm = 1 GROUP BY NC0_5 ORDER BY Count DESC LIMIT 1")
mostCommonNC05: org.apache.spark.sql.DataFrame = [NC0_5: double, Count: bigint]

scala> mostCommonNC05.show()
+-----+
|NC0_5|Count|
+-----+

```

Figure 17: 5.6 & 5.7 Operations

5.8 Most Frequent NC0.5 Value During Alarms

This query found the NC0.5 value that appeared most often when alarms were triggered. The most frequent value was 0.11, occurring 96 times. This insight helps profile the typical air quality during alarm conditions, particularly for smaller particles.

5.9 Possible False Alarms at Low Pollutant Levels

I filtered records to find how many alarms occurred when both PM2.5 and NC1.0 were below 0.05. The result was 1,528 records. These alarms occurred despite very low pollutant levels, suggesting they could be potential false alarms, likely due to sensor sensitivity or other environmental anomalies.

5.10 Fire Alarms at High Ethanol Levels

Finally, I checked how many alarms occurred when Raw Ethanol exceeded 0.5. This condition matched 3,403 records, suggesting that ethanol may occasionally contribute to fire detection. However, the count is relatively small compared to the total alarms, implying that ethanol is a supporting rather than primary factor.

```
scala> mostCommonNC05.show()
+-----+
|          NC0_5|Count|
+-----+
|0.11057242493833461| 96|
+-----+

scala> val falseAlarms = spark.sql(
  """SELECT COUNT(*) AS False_Alarms
    FROM fire_alarm_data
    WHERE Fire_Alarm = 1 AND PM2_5 < 0.05 AND NC1_0 < 0.05"""
)
falseAlarms: org.apache.spark.sql.DataFrame = [False_Alarms: bigint]

scala> falseAlarms.show()
+-----+
|False_Alarms|
+-----+
|          1528|
+-----+

scala> val highEthanolFireAlarms = spark.sql(
  """SELECT COUNT(*) AS High_Ethanol_Fire_Alarms
    FROM fire_alarm_data
    WHERE Fire_Alarm = 1 AND Raw_Ethanol > 0.5"""
)
highEthanolFireAlarms: org.apache.spark.sql.DataFrame = [High_Ethanol_Fire_Alarms: bigint]

scala> highEthanolFireAlarms.show()
+-----+
|High_Ethanol_Fire_Alarms|
+-----+
|                   3403|
+-----+
```

Figure 18: 5.8 & 5.9 & 5.10 Operations

6. Machine Learning Operations

6.1 ML Problem

The chosen machine learning problem is to predict whether a fire alarm will be triggered based on various environmental sensor readings. The dataset includes numerical features such as Raw Ethanol, NC0.5, PM1.0, PM2.5, and NC1.0. The target variable, Fire Alarm, is binary, where a value of 1 indicates that the alarm was activated, and 0 means it was not.

To solve this problem, the selected machine learning algorithm is the Decision Tree Classifier, a popular and interpretable model for binary classification tasks. A decision tree works by recursively splitting the data based on feature thresholds that best separate the classes, ultimately creating a tree-like model that can be used for prediction.

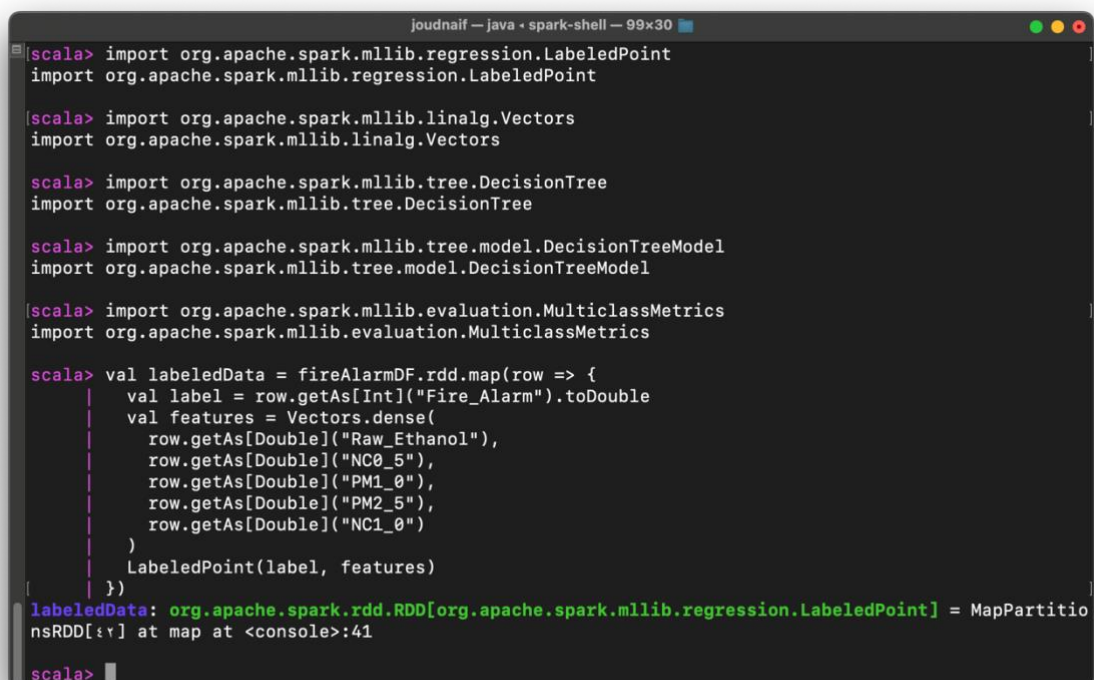
The Decision Tree Classifier is well-suited for this problem because it can handle continuous numerical inputs and is capable of learning non-linear patterns that may exist between the pollutant levels and alarm activations. This is important in a fire detection context, where multiple sensor values may jointly influence whether a fire condition is detected.

The model will be trained using labeled data, where each record includes both sensor readings and the known alarm status. The goal is for the model to learn the underlying patterns that lead to an alarm and to accurately predict future cases based on similar input readings.

By analyzing the trained model, it will also be possible to identify which environmental features (such as NC1.0 or PM2.5) are most influential in predicting fire alarms. This insight can be useful in refining sensor configurations and improving the accuracy of real-world fire detection systems.

6.2 Importing Libraries and Preparing the labelled Data

The machine learning process begins by importing the necessary MLlib classes, including LabeledPoint, Vectors, DecisionTree, and evaluation tools. The dataset, which is already loaded into a Spark DataFrame named "fireAlarmDF," is transformed into an RDD of LabeledPoint objects. The feature columns used are Raw Ethanol, NC0.5, PM1.0, PM2.5, and NC1.0, while the label is the Fire_Alarm column.



```
joudnaif — java - spark-shell — 99x30
scala> import org.apache.spark.mllib.regression.LabeledPoint
import org.apache.spark.mllib.regression.LabeledPoint

scala> import org.apache.spark.mllib.linalg.Vectors
import org.apache.spark.mllib.linalg.Vectors

scala> import org.apache.spark.mllib.tree.DecisionTree
import org.apache.spark.mllib.tree.DecisionTree

scala> import org.apache.spark.mllib.tree.model.DecisionTreeModel
import org.apache.spark.mllib.tree.model.DecisionTreeModel

scala> import org.apache.spark.mllib.evaluation.MulticlassMetrics
import org.apache.spark.mllib.evaluation.MulticlassMetrics

scala> val labeledData = fireAlarmDF.rdd.map(row => {
  |   val label = row.getAs[Int]("Fire_Alarm").toDouble
  |   val features = Vectors.dense(
  |     row.getAs[Double]("Raw_Ethanol"),
  |     row.getAs[Double]("NC0_5"),
  |     row.getAs[Double]("PM1_0"),
  |     row.getAs[Double]("PM2_5"),
  |     row.getAs[Double]("NC1_0")
  |   )
  |   LabeledPoint(label, features)
  | })
labeledData: org.apache.spark.rdd.RDD[org.apache.spark.mllib.regression.LabeledPoint] = MapPartitio
nsRDD[1] at map at <console>:41

scala>
```

6.3 Model Training

After transforming the dataset into labeled points, the next step involved dividing it into training and test sets to facilitate model evaluation. Using the `randomSplit()` function in Spark, 70% of the data was allocated for training and 30% for testing. This split ensures that the model learns from a substantial portion of the dataset while preserving enough records to assess its performance on unseen data.

Following the split, a Decision Tree Classifier was trained using the `trainClassifier` method from Spark's MLlib. As this is a binary classification problem, the number of classes was set to 2. The Gini impurity criterion was selected to measure the quality of splits, as it effectively separates the classes during the tree-building process.

The Decision Tree was configured with the following hyperparameters:

- `maxDepth = 5`
- `maxBins = 32`
- `impurity = "gini"`

A maximum depth of 5 restricts the tree from becoming overly complex, while a higher number of bins (32) allows for more precise handling of continuous features.

The resulting model was a Decision Tree of depth 5 with 29 decision nodes. This model captures several conditional rules from the input features to distinguish between fire alarm and non-alarm conditions. Its structure makes it interpretable and efficient for both training and prediction, making it suitable for deployment in real-time or safety-critical systems.


```
joudnaif - java - spark-shell - 99x26

scala> val splits = labeledData.randomSplit(Array(0.7, 0.3), seed = 1234L)
splits: Array[org.apache.spark.rdd.RDD[org.apache.spark.mllib.regression.LabeledPoint]] = Array(MapPartitionsRDD[1] at randomSplit at <console>:41, MapPartitionsRDD[2] at randomSplit at <console>:41)

scala> val trainingData = splits(0)
trainingData: org.apache.spark.rdd.RDD[org.apache.spark.mllib.regression.LabeledPoint] = MapPartitionsRDD[1] at randomSplit at <console>:41

scala> val testData = splits(1)
testData: org.apache.spark.rdd.RDD[org.apache.spark.mllib.regression.LabeledPoint] = MapPartitionsRDD[2] at randomSplit at <console>:41

scala> val model = DecisionTree.trainClassifier(
  |   trainingData,
  |   numClasses = 2,
  |   categoricalFeaturesInfo = Map[Int, Int](),
  |   impurity = "gini",
  |   maxDepth = 5,
  |   maxBins = 32
  | )
model: org.apache.spark.mllib.tree.model.DecisionTreeModel = DecisionTreeModel classifier of depth 5 with 29 nodes

scala>
```

6.4 Model Evaluation and Results

After training the Decision Tree model, its performance was evaluated on the test dataset. Predictions were generated using the `.predict()` method, which produced a label prediction for each test instance. These predictions were then paired with the actual labels to assess the model's accuracy and classification performance.

The evaluation was conducted using the `MulticlassMetrics` class from Spark MLlib, which provides standard metrics for classification tasks. The following key performance indicators were calculated based on the predictions:

- **Accuracy:** 95.94%
- **Precision** (for class 1 - fire alarm): 96.58%
- **Recall** (for class 1): 97.76%
- **F1 Score:** 97.17%

These results show that the model performs strongly in identifying both fire alarms and non-alarm conditions. The high recall value suggests that the model is effective in detecting true fire alarms, minimizing the risk of missing actual fire events. Meanwhile, the high precision indicates that false alarms are relatively uncommon, reducing unnecessary interventions.

The confusion matrix below provides a detailed view of the model's predictions on the test set:

	Predicted : 0	Predicted: 1
Actual: 0	4,258	399
Actual: 1	258	11,267

As shown, the model correctly classified a large majority of the records. There were 258 false negatives (missed alarms) and 399 false positives (alarms triggered when none occurred), which are relatively low given the total sample size.

Overall, the evaluation metrics indicate that the Decision Tree Classifier is well-suited for this binary classification problem, achieving strong predictive performance across all major measures. The model is accurate, reliable, and shows good generalization to new, unseen data.

```
joudnaif -- java - spark-shell -- 99x26
scala> val predictionsAndLabels = testData.map { point =>
  |   val prediction = model.predict(point.features)
  |   (prediction, point.label)
  | }
predictionsAndLabels: org.apache.spark.rdd.RDD[(Double, Double)] = MapPartitionsRDD[v.] at map at <
console>:42

scala> val metrics = new MulticlassMetrics(predictionsAndLabels)
metrics: org.apache.spark.mllib.evaluation.MulticlassMetrics = org.apache.spark.mllib.evaluation.Mu
lticlassMetrics@ff13935

scala> val accuracy = metrics.accuracy
accuracy: Double = 0.9593993325917687

scala> val precision = metrics.precision(1.0)
precision: Double = 0.9657980456026058

scala> val recall = metrics.recall(1.0)
recall: Double = 0.9776138828633406

scala> val f1 = metrics.fMeasure(1.0)
f1: Double = 0.9716700444137812

scala> println(metrics.confusionMatrix)
4258.0  399.0
258.0   11267.0
```

7. Conclusion

In conclusion, my exploration into predicting fire alarm activation using environmental sensor data has provided meaningful insights. By applying a variety of RDD transformations and Spark SQL queries, I was able to uncover important patterns, such as the strong correlation between NC1.0 and PM2.5 levels and fire alarm triggers. These analyses helped guide the feature selection for the machine learning model.

The classification model was built using a Decision Tree Classifier and achieved strong results across all key evaluation metrics, including accuracy, precision, recall, and F1 score. The model performed reliably in identifying both fire alarm and non-alarm conditions, making it a valuable step toward building a data-driven fire detection system.

Overall, this project offers a solid foundation for combining data analysis and machine learning to support more responsive and intelligent alarm systems. Future work could explore additional modeling techniques or real-time applications to further enhance performance.